CMU 15-213 计算机系统导论 - 第二课：位、字节与整数

课程公告

大家好。上次课是课程概述和一些后勤安排，今天我们将进入真正的内容。一般来说，我不会在这门课上花很多时间做公告，但前几节课我想确保大家清楚一些事项。

劳工节和Linux训练营

这个周一是劳工节，没有课，因此也不会有课堂讨论（recitation）。但我们安排了所谓的**Linux训练营**，时间是周一晚上7点。

关于训练营：

- 完全自愿参加
- 如果你有Linux经验或熟悉命令行操作，可以不来
- 内容包括：如何运行代码、编辑文件、移动文件等基本操作
- 也会涵盖Git，因为我们鼓励使用ECE运营的Git服务器

Git服务器状态： 有些同学反映遇到问题。老实说，系统还没完全设置好。据我了解，ECE负责这个的人在参加"火人节"（Burning Man），他每年都去，每年都有同样的问题。希望他在那里玩得开心，回来后精神焕发，把系统修好，希望在训练营之前就能准备就绪。

晚间复习课

本学期我们会进行几次这样的晚间复习课：

- 都不是强制性的，是为了帮助大家
- 可能会录像，也可能不会
- 所有材料（幻灯片、代码示例等）都会发布在网站上

Lab 0 - 新实验

如我上次提到的，有一个新实验叫**Lab 0**：

- 目的：C编程基础演练
- 如果你已经相当熟悉C编程，应该很简单
- **时间：** 应该只需1-2小时
- **截止日期：** 从今天起一周后的午夜

重要信息：

- 不需要Autolab账户

- 不需要GitHub账户
- 只需要访问网站上的文件和能编译C代码的机器
- 我在Mac和Linux上测试过
- Windows可能不能直接运行（使用了Linux库），但可以用Cygwin创建类Unix环境

评分政策： 这个作业不适用宽限日（grace days）和延期政策。要么完成，要么不完成。这更像是一个评估，让你了解自己是否具备本课程期望的最低编程背景。

关于候补名单

对于还没正式选上课或在候补名单上的同学：

- **请不要联系我们教师关于候补名单问题**
- 我甚至不知道候补名单怎么运作
- 我们正在与CS和ECE的课程管理员合作处理
- 他们比我们更有能力处理这些问题

第二章：位、字节与整数表示

接下来三节课将涵盖教材第二章的内容。如果你翻开书数页数，会发现有很多页，所以进度会很快。这些讲座不会完全重复或涵盖书中的每一个细节。

这些内容都不是超级高深的数学，但你必须非常熟悉这些基础材料，才能继续学习课程的其他部分。

1. 比特的历史与意义

比特的诞生

你们成长在一个比特（bit）已经是生活常识的时代。有趣的是，"比特"这个术语实际上是在1948年左右才首次提出的，由一位名叫**克劳德·香农**（Claude Shannon）的人提出。

- 他没有发明比特本身
- 他发明了**信息论**（Information Theory）
- 信息论将信息的最原始单位定义为比特

为什么是二进制?

当然，你知道计算机中的一切都是用1和0来组织和表达的。你有没有想过，为什么要这样做?

答案： 有原因，但这不是唯一的方式。

历史例子：ENIAC

- 第一台大型计算机叫ENIAC
- 使用真空管实现，非常大、重、昂贵
- **用十进制编码**：用10个真空管代表每个数字
- 真空管开或关代表0-9中的哪个值

所以并不是一开始就用二进制（powers of two）作为思考计算机的主导方式。

为什么二进制成为主流?

从系统设计角度看，有几个好理由：

1. 抗噪声能力

- 如果用示波器或其他仪表测量芯片的信号
- 你会看到信号不是完美的方波
- 有抖动、变化和噪声
- 我们构建电路的器件极其微小，响应不是那么"工程完美"

优势： 用二进制（0或1），可以处理大量噪声、不确定性或系统不完美，仍然可靠地提取出0或1的信号。

2. 存储稳定性

- 对于存储信息，可以想象有些电路有反馈环路
- 可以反馈1或反馈0
- 这是非常稳定的值
- 如果试图离散表示更多值，衰减或噪声等会破坏这些信号

所以，用1和0表示一切已经成为常态。

比特没有固有含义

重要概念： 比特或比特表示中没有任何东西告诉你这是：

- 数字
- 字符串
- 浮点数
- 代码指令

比特就是比特 —— 只是0和1的值。**我们如何赋予这些比特意义，决定了我们如何解释它们。**

这是一个一般性的抽象陈述，我们在本课程中会多次回顾，以真正强化这个想法。

比特的不同解释

示例1：整数表示

- 数字15,213可以用一串1和0表示
- 使用二进制（base-2）表示

示例2：小数表示

- 可以用二进制表示小数
- 不再叫小数点（decimal point），而是**二进制点**（binary point）
- 标记在正2次幂权重值（1, 2, 4, 8）和负2次幂权重值（1/2, 1/4, 1/8）之间

示例3：浮点数

- 浮点数15213.0用二进制表示
- 有二进制点
- 乘以2^13（与15-213课程编号无关，只是缩放需要）
- 将1到2之间的数缩放到15,000

关键点： 如果我看到一个比特串，无法事先知道它是什么数字、什么表示。我可以用不同方式解释它。

2. 十六进制表示法（Hexadecimal）

为什么需要十六进制?

你可以用二进制写出所有比特，但当有64位值时，写出一串1和0会很麻烦。

解决方案： 通常我们将它们打包——每次分组4位，代表0到15的值，用**十六进制记数法**的单个数字表示。

十六进制表示规则

C语言表示法：

- 前缀 `0x` 或 `0X`（大小写均可）
- 使用数字0-9加字母A-F（大小写均可）
- 编码16种可能性

示例：15213的十六进制

```
二进制分组：0011 1011 0110 1101
十六进制：   3   B   6   D
结果：0x3B6D
```

我的记忆技巧：

- 0-9很简单
- 知道A是10
- 知道C是12
- 知道F是15
- 其他的在中间插值推断

如果你被困在荒岛上必须用十六进制编码东西……这是个有趣的困境，对吧？

字长（Word Size）

没有统一的标准。人们会说：

- "你的机器字长是多少？"
- "这是32位机器还是64位机器？"

真相： 这些术语不是很有意义。

示例：Intel处理器

- 硬件内置支持64位整数运算、地址等
- 但也可以运行在兼容模式下
- 兼容模式可以追溯到90年代初，只支持32位操作

所以我们谈论64位机器和32位机器，但事实是**同一台机器既是64位又是32位**。

这适用于你现在遇到的大多数机器。这实际上是操作系统、编译器（生成什么代码）和硬件本身的组合，决定了机器类型。

C语言数据类型的大小

大多数C基本数据类型在32位和64位机器上是独立的，**除了**：

- **long**：从4字节变为8字节
- **指针**：这才是真正重要的
  - 32位机器：4字节，约4GB内存范围
  - 64位机器：8字节，范围大得多

这就是为什么现在转向64位的部分原因——很多机器现在有超过4GB的RAM。

3. 位运算（Bit-Level Operations）

布尔代数的历史

还是那位发明信息论的克劳德·香农，1938/39年写了一篇硕士论文，被认为是**有史以来最伟大的硕士论文**。

他做了什么?

- 将**布尔代数**（由19世纪英国数学家George Boole发展）与数字电路设计联系起来
- Boole试图将命题逻辑（关于真假的推理）编码为代数形式
- 使其更像数学而非抽象哲学概念

核心思想:

- 把0想象成假（false）
- 把1想象成真（true）
- 可以代数引入运算符

基本布尔运算

AND（与）： 只有两个输入都为真时才为真

```
A AND B：A和B都必须为真
```

OR（或）： 任一输入为真即为真

```
A OR B：A或B为真（或两者都真）
```

NOT（非）： 翻转真假

```
NOT A：真变假，假变真
```

XOR（异或）： 一个或另一个为真，但不是两者都真

```
A XOR B：独占或，区别于包含或
```

英语中的歧义： 在英语中，"或"可以是包含或排斥的:

- "你想让我砍掉左手还是右手？"

- 有一种隐含的理解——我不会两只手都砍掉

所以在日常话语中，有时"或"是包含的，有时是排斥的。我们要明确区分这两种不同的操作。

香农的贡献

回到香农的时代，还没有晶体管，他们有**机电继电器**。

- 他们用这些构建了非常复杂的系统，比如整个电话交换网络
- 他是第一个提出基本原理的人：可以从这个继电器网络系统地推导出一个表达式，描述何时会从A点到B点建立连接

让人惊讶的是，在他提出这个联系之前，他们是怎么做到的？这是那种一旦建立联系就显而易见的事情，我们无法想象还有其他方式。

位向量（Bit Vectors）

可以从简单的0或1真假值推广到**比特串或位向量**。

概念：

- 计算机上的字有32或64位
- 可以把它们看作不仅仅是数字，而是一串0和1
- 可以对比特串应用相同的操作
- 逐位应用

示例：AND操作

```
 01101001
& 01010101
----------
 01000001
```

列中两个位置都必须为1才能得到1作为结果。

集合表示（用位向量）

实用应用： 非常实际地用于计算机中。

场景： 假设你最多有32个东西，想要表示这32个东西的不同集合或子集。

解决方案：

- 使用长度为32的比特串
- 如果给定元素在集合中，位为1

- 如果不在，位为0

集合操作映射：

- **AND** → 交集
- **OR** → 并集
- **XOR** → 对称差

真实例子：

- 跟踪我们在任何给定时间维护多少个网络连接
- 能够选择性地说："我会接受来自这个连接子集的传入消息"

有一个库可以创建和操作这些集合，但底层只是使用一个或多个比特字来表示它们。

术语：掩码（Mask） 当你有一串1和0时：

- 1是你现在感兴趣的位置
- 0是你不感兴趣的位置
- 用AND操作只查看感兴趣的项

数学基础：Stone表示定理

有一个定理叫**Stone表示定理**，由Stone先生提出：

- 任意集合（必须可数）与该集合的位向量表示之间存在一一对应关系
- 非常实用，也有数学重要性

4. C语言中的位运算

位级运算符

在C语言和大多数现代编程语言中，可以使用这些操作：

```c
&   // 按位AND
|   // 按位OR
^   // 按位XOR（异或）
~   // 按位NOT（取反）
```

Lab 1预告： 下周二发布，你会获得使用这些位级运算符的深入（或痛苦）经验——学会爱它们、恨它们，或至少使用它们。

十六进制运算示例

示例：单字节值（用两个十六进制数字表示）

除非你是十六进制专家，否则可能不能直接计算，比如4的补码是什么。

方法：使用查找表

1. 写出二进制展开：0100
2. 取反：1011
3. 在表中查找：1011 = B

类似地：

- 1110 = E
- 所以 ~0x41 = 0xBE

十六进制的作用：

- 方便的写法
- 不是方便的推理方式
- 推理时最好转换到二进制

快捷技巧： 有人问："能不能直接从15减？"

- 可以！同样的效果
- 如果从全1减去，不会有进位或借位
- 结果正好是补码

这是思考位运算的有用方法。

逻辑运算符 vs 位运算符

常见混淆： C语言中有另一类运算符，看起来很像位运算符，但**非常非常不同**：

```c
&&  // 逻辑AND（不是按位&）
||  // 逻辑OR（不是按位|）
!   // 逻辑NOT（不是按位~）
```

不同的逻辑：

- 只有两个值：**假（false）或真（true）**

- 假的唯一表示：**0**
- 真：任何非零值

行为：

```c
0x41 && 0x69  // 41非零（真），69非零（真）→ 结果为真
              // 但结果总是生成值1（不是位模式）

!0x41         // 41非零 → !41 = 0（假）
!0x00         // 0 → !0 = 1（真）
```

这实际上很有用，我们一直这样做，但要记住底层这是一种古怪的思考方式。

双重否定技巧：

```c
!!x  // 我总说"bang bang"
     // 如果x为0 → 保持0
     // 如果x非零 → 转换为1
```

你会在第一个实验中使用这个技巧。

短路求值（**Short-Circuit Evaluation**）

重要特性：`&&` 和 `||` 都有提前终止特性。

示例：

```c
p && *p  // 如果p是空指针会怎样?
```

发生什么：

- 如果p是空指针（0）
- 返回0
- 不会计算`*p`
- 很好！因为如果p是空指针，`*p`会出错

规则：从左到右只计算表达式中必要的部分来确定真假。

你可能已经知道这个，但这是一个重要特性。我即使现在还会犯这个错误，不是因为不知道区别，而是因为打字不好。

5. 移位运算（Shift Operations）

除非你有相当的经验，否则可能还没花很多时间使用移位操作。

左移（Left Shift）

一般规则：

- 将位向左滑动指定位置数
- 右侧用0填充
- 左端的位移出并消失

示例：

```c
x << 3  // 左移3位
```

右移（Right Shift）

有两种不同的标准约定：

1. 逻辑右移（Logical Shift）

- 将位向右移动
- 丢弃移出的位
- 左侧用0填充
- 自然、直观

2. 算术右移（Arithmetic Shift）

- 将位向右移动
- 复制最左边的位（符号位）
- 示例中用红色显示

为什么有两种?

现在看起来不直观，但当这些表示可能为负值时：

- 算术右移实际上是一个非常有用的操作
- 粗略来说是除以2的幂

- 稍后会详细讲为什么区分这两种

C语言中的移位

```c
<<  // 左移（只有一种方式）
>>  // 右移（算术或逻辑，取决于类型）
```

右移规则：

- 有符号数：通常是算术右移
- 无符号数：总是逻辑右移
- 具体取决于编译器和机器

警告：未定义行为

1. **负数移位：**

```c
x << -1  // 你可能认为等价于 x >> 1
         // 但实际上是未定义的，不会这样工作
```

2. **移位超出范围：**

```c
x << 32   // 对32位数，你可能期望得到全0
x << 100  // 但可能不会，未定义
```

C语言的问题：

- C语言对许多行为是**未定义的**
- 编译器作者可以选择如何实现
- 可能在不同机器、编译器间行为不同
- 这是试图编写跨多个机器和编译器运行的程序时令人不安的特性

Java的区别：

- Java的设计目标之一是确保所有东西都是定义的
- 程序完全可移植

- 但我们这门课不用Java，我们用C和它的怪癖

6. 整数表示

两大类整数

1. 无符号数（Unsigned）

- 介于0和正数之间的值
- 不需要符号表示正负

2. 有符号数（Signed）

- 可能为负或正的数
- 有很多种表示方法
- 最常见的是二进制补码表示（Two's Complement）

无符号数表示

标准二进制表示：

- 一系列位，每个位由2的幂加权
- 直观、简单

示例：**4**位无符号数

```
位:    1 0 1 0
权重:  8 4 2 1
值:    8 + 2 = 10
```

二进制补码表示（**Two's Complement**）

关键区别： 最高有效位（MSB）具有负权重。

公式：

```
值 = -w[n-1] × 2^(n-1) + w[n-2] × 2^(n-2) + ... + w[0] × 2^0
```

其中 w[n-1] 是符号位。

示例**1**：**+10**（**5**位）

```
位:    0 1 0 1 0
权重: -16 8 4 2 1
值:    0 + 8 + 2 = 10
```

符号位为0（正数）。

示例2：-10（5位）

```
位:    1 0 1 1 0
权重: -16 8 4 2 1
值:    -16 + 4 + 2 = -10
```

逻辑：

- 从-16开始（有点太负了）
- 加回4和2得到-10
- 一般方案：负数从很负的值开始，然后加回正的2的幂到我们想要的值

16位表示示例

表示15213和-15213：

```
15213:
位模式: 0011 1011 0110 1101
     0 后跟一堆1

-15213:
位模式: 1100 0100 1001 0011
     1 后跟一堆0
```

关键： 最高有效位权重是 2^15 = 32,768

7. 数值范围

无符号数范围

最小值： 全0 = 0

最大值： 全1

```
UMax = 2^w - 1
```

示例（4位）：

- 最大值：1111 = 15 = 2^4 - 1

一堆1总是几乎等于2的幂，但不完全是。

二进制补码范围

最负值： 只有符号位为1

```
TMin = -2^(w-1)
```

最正值： 符号位为0，其余全1

```
TMax = 2^(w-1) - 1
```

-1： 全1

```
-1 = 1111...1111
```

负值和正值几乎相互抵消，但不完全。

识别负数的技巧： 如果看到十六进制数 `0xFFFF....`（一堆F），很可能是负数：

- 一串1是非常负的值
- 一堆1把它拉回接近0
- 剩下的就是实际值

16位具体范围

无符号：

- 最大值：2^16 - 1 = 65,535

有符号（二进制补码）：

- 最大值：2^15 - 1 = 32,767
- 最小值：-2^15 = -32,768

特殊值：

- **-1：** 总是全1
- **0：** 总是全0

不对称性

重要观察： 存在不对称性。

```
|TMin| > |TMax|
```

作为负数能表示的绝对值比作为正数能表示的要大。

为什么？ 因为需要为0留空间。

- 第一位为1 → 负数
- 第一位为0 → 0或正数

所以有一个"缺失的正数"。

影响： 大多数时候不重要，但当重要时，真的很烦人。

常见字长范围

在几乎所有现代计算机上，可以操作的值是：

- 8位（1字节）
- 16位
- 32位
- 64位

C语言都支持这些数据类型。

32位的影响：

- 2^32 ≈ 4,000,000,000（40亿）
- 我的标准技巧：$\boxed{1024 \approx 1000}$
- 2^32 = 2^30 × 4 ≈ 10^9 × 4 ≈ 40亿
- 不完全准确，但足够量级估算

传闻： 有人说微软切换到64位机器是因为无法用32位数表示比尔·盖茨的净资产。好消息是，64位数还有很长的路要走才会遇到这个问题。

C语言的limits.h

最佳实践： 不要太随意地假设数字范围。

不同机器可能不总是使用二进制补码表示（虽然几乎都用）。

解决方案： C有特殊的包含库文件 $\boxed{\text{limits.h}}$：

- 定义标准数据类型的最大/最小值
- 对于特定机器或编译器应该是正确的

现实： 很多人"作弊"，假设是这些特定值。

8. 无符号与有符号之间的映射

位模式的双重解释

核心概念： 看同一个位模式，可以：

- 解释为无符号数
- 解释为二进制补码数

关系： 这两个数如何相互关联？

观察（绿色部分）： 如果是非负数，无符号和二进制补码表示**相同**。

高位为1时： 有一个奇怪的映射——无符号数和对应的二进制补码数之间有一个增量，正好是16（在这个例子中）。

实时测验

现在让我们尝试一个实验，看看这是否有助于教育进步。我会很惊讶如果这能成功。

指示：

1. 访问课程网页
2. 左侧有Canvas链接（新版Blackboard）
3. 找到第2天的测验

[学生参与测验]

好消息： 你们都做对了，测验答案是错的！

基本上每个人都做得很好。我很高兴看到这些内容大家基本都见过。我假设任何答错的人只是不小心弄错了某个位。

这是一个实验。我知道人们在课堂上成功使用过这项技术，所以我想给它一个机会。

9. C语言中的类型转换

显式转换（Casting）

奇怪的问题： 在C中，如果我有一个数 x：

```
c
```

```
int x = -1;
unsigned ux = (unsigned) x;  // 显式转换
```

问题： ux会是什么值？

可能的想法：

- -1是负数
- 不能表示为无符号数
- 给你0，只差1，应该有部分分数？

实际发生什么： 在大多数机器上是什么？

答案： 2^32 - 1（或2^31，取决于字长）

为什么？ 想象一个全1的无符号数——这就是-1的样子。

关键规则： 在C中，在有符号和无符号之间转换时，**位保持不变**。

只是突然间，我把那个高位看作正权重或负权重，取决于我是把它看作有符号还是无符号。

与浮点转换的对比

重要区别：

```c
float f = 3.14;
int i = (int) f;  // 实际改变位模式
```

如果说某个东西是浮点数，然后转换为int，那么**真的会发生变化**——位真的会改变。

这是一个相当重要的概念。但现在我们只记录这个特殊情况：在无符号和有符号之间，位保持不变。

这在C和大多数支持无符号数据类型的语言中都是真的。

转换的逻辑视图

概念上：

- 我们要把一个可能为负的数
- 变形为一个肯定不为负的数
- 规则是保持位相同
- 真正改变的只是相关的数值

反过来也一样： 无符号到有符号。

查表示例

回到我们的小表，左右看：

- 如果两者都在高位为0的范围内 → 数值相同
- 如果在高位为1的范围内 → 增加或减少16（或2^w）

图示视图：

- 绿色范围的数（较小量级）→ 保持不变
- 以前非常负的数 → 现在变成非常正的数
- 反方向也一样

这实际上可能导致一些奇怪的行为，如果不了解或没有准备，可能导致非常不愉快的结果。

Java的选择： 由于这种危险，Java甚至没有无符号数据类型。

10. 隐式转换

C语言的隐式转换

可以显式转换，但也有**隐式转换**：

```c
int tx, ty;
unsigned ux, uy;

tx = ux;  // 隐式转换：unsigned → signed
uy = ty;  // 隐式转换：signed → unsigned
```

发生什么： 位保持不变，但逻辑上在无符号和有符号数之间转换。

更隐蔽的情况：

```c
unsigned get_unsigned();
void takes_unsigned(unsigned u);

int x;
unsigned y = get_unsigned();
x = y;                  // 隐式转换
takes_unsigned(x);      // 隐式转换
```

混合运算中的转换

奇怪的规则： 如果在任何操作中一个参数是有符号的，另一个是无符号的：

规则：自动将有符号数转换为无符号数，然后继续操作。

这适用于：

- 加法
- 乘法
- **比较运算**（<, >, <=, >=, ==, !=）

令人惊讶的比较结果

重要提示： C中写的任何十进制数默认是**有符号的**，除非在末尾加U或u。

示例分析：

```c
0 == 0U  // 相等
```

- 两者都是0
- 位模式都是全0
- 对于接近0的所有数，无符号和二进制补码表示没有区别
- **结果：相等 ✓**

```c
-1 < 0  // 小于
```

- 显然，负数小于0
- **结果：小于 ✓**

```c
-1 < 0U  // ? ? ?
```

- 右边是无符号的（有U）
- **隐式转换：**-1转换为无符号数
- -1变成一个非常大的数（全1）
- **结果：大于 ✗**（令人惊讶！）

```c
2147483647 > -2147483648  // TMax > TMin
```

- 左边：0后跟一堆1（TMax，最大正数）
- 右边：1后跟一堆0（TMin，最负数）
- **结果：大于（正数大于负数）✓**

```c
2147483647U < -2147483648  // 加了U
```

- 现在是无符号比较
- 左边（无符号）：0后跟一堆1 → 较小
- 右边（转换为无符号）：1后跟一堆0 → 较大
- **结果：小于✗（令人惊讶！）**

[画图解释]

```
位模式：
0111...1111  (TMax)
1000...0000  (TMin)

无符号比较：左边较小
有符号比较：左边（正数）较大
```

更多示例

```c
-1 > -2        // 大于✓
(unsigned)-1 > -2  // 大于✓（两者都转为无符号，都是大数）

2147483647 > (int)2147483648U  // 大于
// 2147483648U转为int变成TMin（-2147483648）
```

要点： 这些都是这种转换规则的实例。

虽然这些看起来很抽象——是很好的考试题材——但这在真实程序中确实是一个风险。当你开始使用无符号数时，有些你习惯的东西不再适用。

11. 无符号数的陷阱

示例1：倒数循环

```c
unsigned i;
for (i = cnt-2; i >= 0; i--)
    a[i] += a[i+1];
```

问题：这个程序有什么问题?

严重问题： 这是一个无限循环。

为什么?

- i是无符号的
- 无符号数总是 >= 0
- i会到达0
- 然后递减0
- 无符号数递减0会得到什么?

答案： 2^32 - 1（或2^64 - 1）—— 一堆1。

结果：

- a[i]会在一个实际上不存在的地址（非常大的数）
- 可能在任何机器上都无效的内存区域
- 会触发**段错误**（segmentation fault）

所以会崩溃而不是无限循环，但无论如何都不会按预期工作。

你看这个会想："傻瓜，你当然不能说 i >= 0，永远不会发生。"

示例2：更微妙的错误

但可能比这微妙得多：

```c
```

```
unsigned i;
int delta = ...;  // 某个正数

for (i = cnt-delta; i < cnt; i++)
    a[i] += a[i+delta];
```

看起来合理：

- delta是正数
- 不会让i低于0
- i一开始是有符号int

有人想猜这段代码会怎样？

问题： 同样的问题！

为什么？

关键： sizeof运算符

- 在C中，sizeof是一个非常重要的运算符
- 返回类型为 size_t
- size_t在所有机器上定义为无符号数
- 足够大，可以容纳数据类型的任意范围值
- 32位运行时：2^32范围
- 64位运行时：2^64范围

问题所在：

```c
int delta = sizeof(...);  // delta是无符号的！
```

即使我们声明为int delta，如果它来自sizeof，它实际上是无符号的。

当我从i（有符号）减去delta（无符号）时：

- 发生隐式转换
- 结果是无符号的
- 即使i开始是有符号int，减去后结果变成无符号

同样的问题： 永远不会小于0，进入同样的问题。

要点： 这类bug你可能会盯着看几个小时，因为很难记住这个语言的特殊怪癖。

这些是特别难处理的bug类型。

---

12. 改变位宽度

到目前为止，我们已经看到一些东西是8位、16位、32位或64位。所以有很多操作，出于某种原因，必须在这些不同大小之间转换。

符号扩展（Sign Extension）

从小表示到大表示的一般规则： 对有符号数做**符号扩展**。

符号扩展的想法：

- 从较小字长扩展到较大字长时
- **不断复制最左边的（符号）位**

为什么这样做？ 在二进制补码表示中，这会**保持数值**。

符号扩展示例

5位到6位扩展：

正数：

```
5位：0 1010（值：10）
6位：00 1010（值：10）
```

只是添加一个0。显然不改变值。

负数：

```
5位：1 0110（值：-10）
6位：11 0110（值：-10）
```

逻辑：

- 5位数中，最高有效位权重为-16
- 6位数中，那个位变成+16
- 但我添加了一个新位，权重为-32
- 如果两者都是1：-32 + 16 = -16（抵消）

一般原理：

- 引入一串位
- 它们的总权重加在一起
- 等于早期数字中单个位的权重

扩展任意长度： 可以向左符号扩展任意长度，得到相同的数值。

C语言中的应用： 当你从 `char` 转换到 `int` 时，会进行符号扩展以保持值。

截断（Truncation）

反方向： 从大数缩小到小数。

第一反应： "等等，你不能那样做！香农的信息论告诉我有一定量的信息，不能变得更小。"

真相： 这是对的。我们**可能会丢失信息**。甚至更糟：不仅可能丢失东西，还可能得出与早期值看似毫无关系的东西。

截断示例

5位到4位（有符号）：

接近零的数：

```
5位：0 1010（10）
4位：  1010  （10）
```

如果数字足够接近零，可以丢掉最高有效位，仍然保持数值。（符号扩展的逆操作）

超出范围的数：

```
5位：0 1010（10）
4位：  1010  （-6）
```

- 原来是10（正数）
- 现在变成-6
- **减少了16**

为什么?

- 原来4位时最高位权重是+8
- 现在变成-8
- 不再相同值
- 无法用4位二进制补码表示10（范围：-8到+7）

负数示例：

```
5位：1 0110  (-10)
4位：  0110  (6)
```

- 原来-10
- 变成+6
- 翻转了符号

要点： 当丢失信息时，只是改变如何看待这些位，可能或可能不是正确的事情，除非数字已经相当接近零。

总结

今天就到这里。

核心概念回顾：

1. **位没有固有含义** —— 我们赋予它们意义
2. **十六进制是方便的记法**，但推理时用二进制
3. **位运算**在集合表示等方面非常实用
4. **无符号与有符号**有不同的表示和范围
5. **二进制补码**是有符号数的标准表示
6. **类型转换**（显式和隐式）可能导致意外结果
7. **符号扩展和截断**有明确的规则和后果

下节课预告： 我们会继续深入这些主题，下周这个时候讨论浮点数。

重要提醒： 这些都是基础材料。你必须非常熟悉才能继续学习课程的其他部分。虽然不是超级高深的数学，但需要真正掌握。

祝大家学习顺利！